PRODUCT REQUIREMENTS DOCUMENT (PRD)

CaféNest — Cafe Management and Online Ordering System

Document Version: 1.0
Project Type: College DBMS Lab Project
Team: Om Watane, Ambika Shelke, Tejaswani, and one additional team member
Technology Stack:

- Frontend: HTML, CSS, JavaScript (vanilla JavaScript only)
- Backend: Python
- Database: SQL
- Version Control: Git and GitHub

Mandatory Technology Restriction: Do not use PHP, Node.js, Express.js, Java, C#, TypeScript, React, Angular, Vue, or any other prohibited programming language or frontend framework. All server-side logic and APIs must use Python. All persistent application data must be stored in an SQL database.

Prepared as a project planning document. Requirements and technical details should be checked against the final implementation before submission.

---

1. Executive Summary

CaféNest is a web-based cafe management and online ordering system designed to bring customer ordering and day-to-day cafe operations into one place. Customers can explore the menu, add items to a cart, place orders, select a simulated payment method, and view order status. Cafe staff and administrators can manage menu items, orders, tables, stock, suppliers, staff information, and reports.

The project is intended for a college DBMS lab demonstration. It combines a customer-facing ordering experience with an administrative interface and a relational SQL database.

The frontend will use HTML, CSS, and JavaScript. The backend will use Python to validate requests, enforce business rules, and communicate with the SQL database.

2. Problem Statement

Small cafes may handle orders, menu updates, stock tracking, and daily reporting manually or through separate tools. This can lead to order-entry mistakes, delayed updates, poor visibility of stock, and time-consuming record keeping.

CaféNest aims to provide a single, straightforward system for these activities while demonstrating database design, CRUD operations, validation, and basic reporting.

3. Product Vision and Objectives

- Provide a clear and responsive online menu and ordering flow.
- Reduce manual work involved in recording orders and updating order status.
- Help administrators maintain menu items, categories, stock, suppliers, and staff records.
- Provide useful summaries such as order totals, popular items, and low-stock items.
- Demonstrate a connected HTML/CSS/JavaScript frontend, Python backend, and SQL database.

- Use simulated payment confirmation for the college demonstration; do not process real payments.

4. Target Users

4.1 Customers

- Browse food and beverage items, categories, prices, and availability.
- Register or log in, manage a cart, submit orders, and review order history.
- Choose a demonstration payment option such as Cash or simulated UPI.

4.2 Cafe Staff

- View incoming orders and update preparation or fulfillment status, subject to assigned permissions.
- Review relevant table and order details.

4.3 Administrator

- Manage menu, categories, staff, tables, inventory, suppliers, and order records.
- Review dashboard metrics and operational reports.

5. Functional Requirements

5.1 Customer Accounts

- Provide registration and login forms with input validation.
- Store customer details and password hashes securely.
- Allow customers to view or update appropriate profile details.

5.2 Menu and Categories

- Display menu items with name, description, image, price, category, and availability.
- Allow authorized administrators to add, edit, deactivate, or remove menu items and categories.
- Show clear feedback when an item is unavailable.

5.3 Cart and Checkout

- Allow customers to add items, change quantities, and remove items from the cart.
- Calculate subtotal and any configured charges consistently on the Python backend.
- Validate cart contents and item availability before creating an order.
- Display an order confirmation and reference number after successful placement.

5.4 Orders

- Save order headers and their individual line items in SQL.
- Support appropriate statuses such as Pending, Confirmed, Preparing, Ready, Completed, or Cancelled.
- Allow customers to view their own order history and current status.
- Allow authorized staff/admin users to view and update orders.

5.5 Payments

- Provide Cash and simulated UPI as demonstration payment methods.
- Record payment method, amount, and simulated status in SQL.

Payment scope: No real UPI transaction, payment gateway, or actual money transfer is included.

5.6 Tables and Cafe Service

- Maintain cafe table numbers and availability/status.
- Associate a table with an order when the dine-in flow is enabled.

5.7 Inventory and Suppliers

- Maintain stock item names, quantities, units, and low-stock thresholds.
- Record stock additions or deductions as stock movements.
- Maintain supplier contact details and related inventory records.
- Show low-stock warnings when quantities reach configured thresholds.

5.8 Staff and Administration

- Maintain staff details and roles.
- Restrict administrative actions to authorized users.

5.9 Dashboard and Reports

- Show summary figures such as order count, sales total, pending orders, and low-stock items.
- Provide date-filtered order/sales summaries where implemented.
- Show popular menu items or other useful trends based on stored data.

5.10 Notifications and Feedback

- Show clear success, validation, and error messages.
- Display order status updates within the application.
- Email/SMS notifications are optional future enhancements unless explicitly implemented.

6. UI and Usability Requirements

- Use a consistent visual style, readable typography, and clear navigation.
- Provide separate customer and administrator entry points.
- Use food and cafe imagery that loads correctly and has suitable fallback behavior.
- Make key pages usable on desktop and mobile-sized screens.
- Show loading, empty, success, and error states where relevant.
- Avoid placeholder or decorative buttons that do not perform their stated action.

7. Non-Functional Requirements

7.1 Performance

Common actions such as opening the menu, adding to cart, and loading orders should respond promptly under the expected classroom demonstration load.

7.2 Security

- Validate input on both client and server.
- Use secure password hashing and session checks for authenticated routes.
- Use parameterized SQL queries and restrict access to records by role.
- Do not store plain-text passwords or sensitive payment information.

7.3 Reliability and Data Integrity

- Prevent invalid quantities and inconsistent order totals.
- Use database transactions for multi-step order placement and related updates.
- Show understandable errors without exposing server secrets.

7.4 Maintainability

Keep frontend pages, Python backend modules, API endpoints, shared configuration, and SQL database scripts organized into clear folders.

8. Project Scope

8.1 Included

- Customer menu, cart, checkout, order history, and status display.
- Admin/staff operations for menu, orders, tables, staff, inventory, and suppliers as implemented.
- SQL-backed records, dashboard summaries, validation, and simulated payment flow.

8.2 Excluded from the Initial Version

- Real payment processing and actual UPI integration.
- Production-grade delivery logistics or third-party courier integration.
- Native Android/iOS applications.
- Automated email/SMS services unless added separately.

9. Success Criteria and Acceptance

- A customer can browse available menu items and complete a valid order.
- Cart quantities and totals are calculated correctly and revalidated by the Python backend.
- Order records and order line items are saved consistently in SQL.
- A user can see appropriate order status and order history.
- Authorized administrators can update menu items and review orders.
- Inventory and low-stock views show the saved data accurately when those modules are enabled.
- Invalid inputs and unauthorized access are handled safely.
- The project runs on the intended demonstration environment and its main links and buttons work.

10. Deliverables

- Working CaféNest web application.
- HTML, CSS, and JavaScript frontend source files.
- Python backend modules and API endpoints.
- SQL database schema and sample data scripts.
- Test cases and evidence of key workflows.
- Project documentation, including PRD and TRD.
- GitHub repository and setup instructions.

11. Assumptions and Dependencies

The required implementation uses HTML, CSS, JavaScript, Python, and SQL. The SQL database engine and Python web framework must be confirmed before implementation if they are not specified in the approved project setup.

PHP and other prohibited technologies must not be introduced. If any older project document mentions PHP, that reference is superseded by this corrected technology stack.

All feature statements describe intended requirements, not proof that a feature is already complete. Final acceptance should be based on testing the actual build.

12. Mandatory Technology Stack Rules

1. **Frontend:** HTML, CSS, and vanilla JavaScript only.
2. **Backend:** Python only for server-side logic and APIs.
3. **Database:** SQL for persistent application data.
4. Do not use PHP, Node.js, Express.js, Java, C#, TypeScript, React, Angular, Vue, or other prohibited technologies.
5. Do not use JSON files, browser localStorage, sessionStorage, or temporary in-memory data as a replacement for the permanent SQL database.
6. Do not create a second backend in another language.
7. Do not silently change the technology stack.
8. All features must be implemented, connected, and tested before being marked complete.

Final PRD Definition: CaféNest is a cafe management and online ordering system with customer ordering, menu management, order processing, simulated payments, cafe table management, staff records, supplier management, inventory tracking, and administrative reporting. The required stack is HTML, CSS, JavaScript, Python, and SQL.